

Monte Carlo Counterfactual Regret Minimization

May 8, 2026

Abstract

This report studies Monte Carlo Counterfactual Regret Minimization (MCCFR), focusing on external and outcome sampling. The project builds on my previous implementation and report on regret matching and full-tree CFR for Kuhn Poker [1]. It asks what changes when the regret update is estimated from sampled trajectories instead of a complete traversal of the game tree. I keep Kuhn Poker as a sanity check, but use Leduc Poker as the main experiment because it is large enough to show the computational motivation for sampling. I implement full-tree CFR, external-sampling MCCFR, and outcome-sampling MCCFR in Python. Outcome sampling is especially relevant to this course because it uses importance weighting to correct for sampling only one terminal trajectory.

1 From CFR to MCCFR

Counterfactual Regret Minimization (CFR) is a regret-minimization method for extensive-form games with imperfect information. The key idea is to place a small regret minimizer at every information set. At each iteration, the algorithm computes a counterfactual value for every legal action at each information set, updates local regrets, and then obtains the next behavioral strategy by regret matching.

In the full-tree version of CFR, the update is exact for the current strategy profile because it traverses the entire game tree. This is very clear pedagogically, but it is also the bottleneck: one iteration costs time proportional to the size of the tree. Kuhn Poker is tiny, so this is not an issue there. Leduc Poker is still small, but large enough that full tree traversal becomes noticeably more expensive. In larger games such as poker variants with more cards, betting actions, or public states, a full traversal can become too expensive.

MCCFR keeps the same local regret-minimization structure but estimates the counterfactual update by sampling part of the tree. One version studied here is *external sampling*. When updating player i , chance nodes and opponent nodes are sampled, while player i 's own decision nodes branch over all legal actions. Therefore, when the sampled path reaches an information set of player i , the algorithm can still compare all actions at that information set:

$$R_i^T(I, a) \leftarrow R_i^{T-1}(I, a) + \hat{v}_i(I, a) - \hat{v}_i(I),$$

where the hats indicate sampled estimates. The important point is that this has the same form as the CFR regret update, but the values are computed using a sampled subtree instead of the whole tree.

The other version is *outcome sampling*. It goes further and samples the update player’s action too, so one update sees essentially one terminal trajectory. Since unchosen actions are no longer observed, the estimator needs importance weights based on the probability of the sampled trajectory. This makes outcome sampling more directly connected to Monte Carlo simulation ideas, but also gives it higher variance.

2 Leduc Poker as the Test Game

Leduc Hold’em is a small poker benchmark introduced in the opponent modeling literature [2]. It is larger than Kuhn Poker but still small enough to evaluate exactly in a class project. The deck contains six cards:

$$J, J, Q, Q, K, K.$$

Each player antes one chip and receives one private card. After the first betting round, one public card is revealed. A second betting round follows, and if no one folds, the hand goes to showdown.

The game contains the main modeling features that make poker interesting:

- private information, because each player only observes their own card;
- sequential decisions, because betting history affects later actions;
- information sets, because the same private card and public history can arise under different hidden opponent cards;
- public chance, because the board card changes hand strength after the first betting round;
- bluffing, because weak hands can represent strong future or made hands.

The fixed-limit version used here follows the common benchmark formalization. The round-1 bet size is 2, the round-2 bet size is 4, and each round has a maximum of two bet/raise actions. At showdown, a pair with the public card beats any high card. If neither player pairs the public card, the higher private rank wins, with $K > Q > J$. Equal private ranks split the pot.

In the code, the game is represented as a simple dictionary of nodes. A chance node stores either the initial ordered private-card deal or the public-card deal. A decision node stores the acting player, the information set, and the legal actions. A terminal node stores the zero-sum utility vector. With the above parameters, the generated Leduc tree has 9,451 nodes and 288 information sets, compared with only a few dozen strategically relevant nodes in Kuhn Poker. This makes Leduc a better setting for seeing why MCCFR is useful.

3 Regret Matching and Full-Tree CFR

Regret matching is the local learning rule used in this project. Suppose an information set has action set A . Let $R^T(a)$ be the cumulative regret for action a after T updates. Regret matching chooses the next mixed action by normalizing only the positive regrets:

$$\sigma^{T+1}(a) = \begin{cases} \frac{[R^T(a)]_+}{\sum_{b \in A} [R^T(b)]_+}, & \text{if } \sum_{b \in A} [R^T(b)]_+ > 0, \\ \frac{1}{|A|}, & \text{otherwise.} \end{cases}$$

The uniform fallback matters early in training because all regrets start at zero.

In full-tree CFR, each information set has its own regret table. At each iteration and for each update player i , the algorithm recursively traverses the full game tree. Let I be an information set of player i , and let $a \in A(I)$. The update has the form

$$R_i^T(I, a) \leftarrow R_i^{T-1}(I, a) + \pi_{-i}^{\sigma^T}(I) (v_i(\sigma_{I \rightarrow a}^T, I) - v_i(\sigma^T, I)).$$

Here $\pi_{-i}^{\sigma^T}(I)$ is the counterfactual reach probability from chance and the opponent, $v_i(\sigma_{I \rightarrow a}^T, I)$ is the counterfactual value of deviating to action a at I , and $v_i(\sigma^T, I)$ is the value of following the current mixed action at I .

For implementation, this can be written more concretely. At an update player node, recursively compute one child value for each legal action:

$$u(a) = \text{value returned by the child after action } a.$$

The value of the current mixed action is

$$u(\sigma) = \sum_{a \in A(I)} \sigma(I, a) u(a).$$

Then the regret increment is proportional to $u(a) - u(\sigma)$. The counterfactual reach factor is what makes the local regret update consistent with the global extensive-form game.

The full-tree implementation in this project keeps two tables: cumulative regrets and cumulative average-strategy weights. The regret table determines the current strategy through regret matching. The strategy-sum table is used at the end to return the average strategy, which is the strategy that converges in the standard CFR guarantee.

4 External Sampling

The implementation uses the following recursive view. At a terminal node, the recursion returns the payoff of the update player. At a chance node, one chance outcome is sampled. At an opponent node, one opponent action is sampled from the opponent's

current regret-matching strategy. At an update player node, every legal action is recursively evaluated, and the local regret increment is

$$\widehat{v}(a) - \sum_{b \in A(I)} \sigma(I, b) \widehat{v}(b).$$

This is the same action-value minus node-value form used in ordinary CFR.

The average strategy is stored in a tabular strategy-sum table. For external sampling, I use the simple sampled averaging rule: when an opponent information set is visited during a sampled traversal, the opponent's current strategy at that information set is added to the corresponding strategy sum. Each player is the opponent once per MCCFR iteration because the implementation performs one update for player 0 and one update for player 1.

5 Outcome Sampling and Importance Weights

Outcome sampling samples chance, opponent actions, and the update player's own actions. Thus one recursive call follows a single terminal trajectory instead of branching at the update player's information sets. This makes each iteration cheaper, but the algorithm must correct for the fact that most actions are not observed.

Let $\sigma(I)$ be the regret-matching strategy at information set I . For the update player, actions are sampled from an exploration policy

$$q(I, a) = (1 - \epsilon)\sigma(I, a) + \epsilon \frac{1}{|A(I)|}.$$

In the experiments I use $\epsilon = 0.6$, matching the common vanilla outcome-sampling choice. If action a^* is sampled at I , the sampled child value is corrected by

$$\widehat{u}(a^*) = \frac{u(a^*)}{q(I, a^*)}, \quad \widehat{u}(a) = 0 \quad \text{for } a \neq a^*.$$

The regret update then uses

$$\frac{\pi_{-i}(I)}{q(h)} \left(\widehat{u}(a) - \sum_{b \in A(I)} \sigma(I, b) \widehat{u}(b) \right),$$

where $q(h)$ is the probability of sampling the trajectory prefix leading to I . This is the importance-sampling part of the algorithm: sampled events that were unlikely under q receive larger weight. The estimator targets the same counterfactual-regret quantity as CFR, but its variance can be much larger.

6 Implementation

The code is self-contained and intentionally small. It follows the direct tabular style of my earlier CFR repository [1], but does not import that code. The Kuhn Poker

game is represented as a dictionary of extensive-form nodes. Chance nodes store the six equally likely card deals, decision nodes store player, information set, and action list, and terminal nodes store the zero-sum utility vector.

All three training algorithms share the same basic interface:

```
train(iterations),    current_strategy(),    average_strategy().
```

The full-tree CFR class keeps the standard counterfactual reach-probability weights. The MCCFR class keeps the same regret tables but only traverses a sampled part of the tree during each update. I also implemented exact evaluation helpers for expected value, best response value, and exploitability. For exploitability, the behavioral strategy is converted to sequence-form realization weights and then evaluated against exact best responses.

The main source files are organized as follows:

- `kuhn_poker.py`: explicit Kuhn Poker game tree used for sanity checks;
- `leduc_poker.py`: generated fixed-limit Leduc Poker game tree;
- `regret_matching.py`: positive-regret normalization with a uniform fallback;
- `cfr.py`: full-tree CFR baseline;
- `external_sampling_mccfr.py`: external-sampling MCCFR;
- `outcome_sampling_mccfr.py`: outcome-sampling MCCFR with epsilon exploration and importance weights;
- `util.py`: expected value, best response, exploitability, and strategy formatting helpers.

Exploitability is computed exactly rather than estimated from rollouts. For each player, the best-response recursion carries distributions over histories weighted only by chance and opponent reach. When the best-response player acts, histories are grouped by information set, so one action is chosen for the whole information set rather than separately at each hidden state. The reported exploitability is

$$\frac{1}{2} (\text{BR}_0(\bar{\sigma}_1) + \text{BR}_1(\bar{\sigma}_0)),$$

which is NashConv divided by two for a two-player zero-sum game.

I also count how many tree nodes are visited during regret updates. This counter is not needed by the algorithm, but it helps explain the difference between full-tree CFR and MCCFR. Full CFR visits the whole tree on each update. External sampling visits only a sampled part of the tree, although it still branches over the update player's own actions.

7 Tests and Sanity Checks

The project includes small tests because implementation errors in CFR can be subtle. The tests check:

- the Kuhn Poker chance node has six uniform deals;
- the Leduc Poker chance node has thirty uniform private-card deals;
- all terminal utilities are zero-sum;
- regret matching normalizes positive regrets correctly and falls back to the uniform strategy when needed;
- exploitability is finite for a known simple strategy;
- full-tree CFR converges near the Kuhn Poker value after 10,000 iterations;
- outcome-sampling MCCFR converges near the Kuhn Poker value after 100,000 iterations;
- the generated Leduc game has finite uniform-strategy exploitability;
- external-sampling MCCFR reduces exploitability over multiple fixed random seeds;
- MCCFR visits fewer nodes per iteration than full-tree CFR.

These tests are not meant to prove correctness, but they catch the most common mistakes: wrong terminal payoffs, incorrect information-set action lists, missing uniform fallback, and regret updates with the wrong sign.

8 Experiments

The main experiment compares full-tree CFR, external-sampling MCCFR, and outcome-sampling MCCFR on Leduc Poker. I ran full-tree CFR for 500 iterations, external sampling for 100,000 iterations, and outcome sampling for 300,000 iterations. The MCCFR numbers below are averaged over three random seeds.

Algorithm	Iterations	Value	Exploitability	Regret-update nodes
CFR	500	-0.091170	0.030586	9.45×10^6
ES-MCCFR	100,000	-0.088230	0.041565	4.23×10^6
OS-MCCFR	300,000	-0.093453	0.237494	4.13×10^6

All three methods reduce exploitability from the uniform-strategy baseline, but the sampling schemes behave differently. CFR has the smoothest curve because every update is exact for the current profile. External sampling is noisier, but still stable because the update player branches over all actions at visited information sets. Outcome sampling

is the noisiest: it uses importance weighting to learn from one sampled trajectory, so its variance is visibly larger.

This node-count comparison is the main reason MCCFR is interesting. In this implementation, full CFR visits about 18,902 tree nodes per iteration, external sampling visits about 42.6 nodes per iteration, and outcome sampling visits about 13.7 nodes per iteration. Outcome sampling is therefore the cheapest per iteration, but in this experiment it needs many more samples to overcome variance.

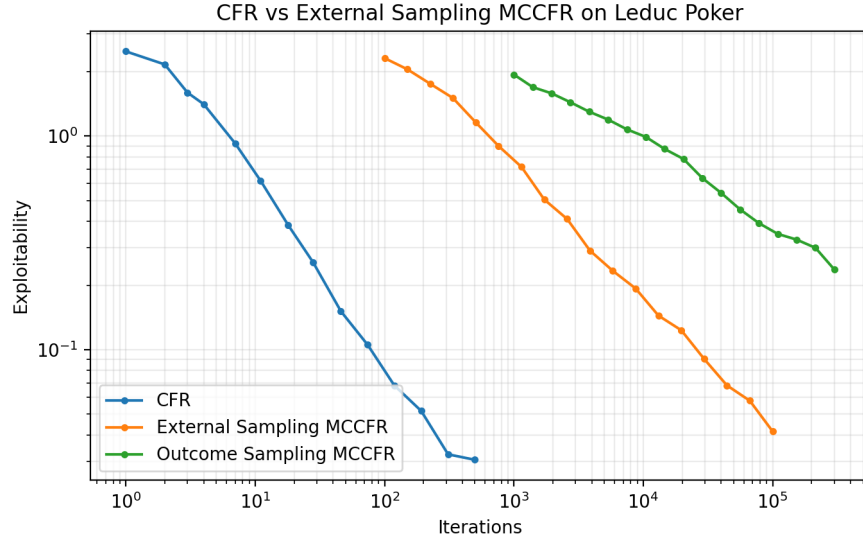


Figure 1: Exploitability of full-tree CFR, external-sampling MCCFR, and outcome-sampling MCCFR on Leduc Poker.

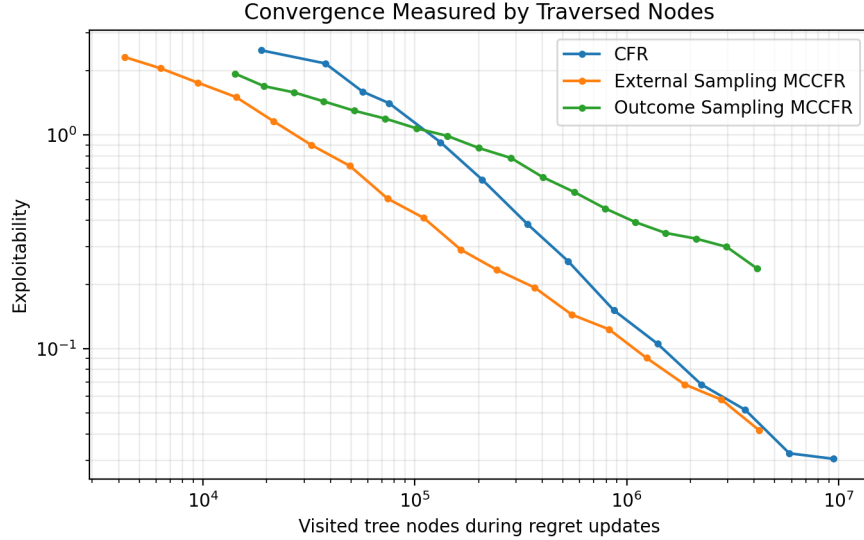


Figure 2: The same convergence curves measured by the number of tree nodes visited during regret updates.

The external-sampling learned strategies also match the qualitative structure of Leduc Poker. This is a useful sanity check because a small exploitability number alone does not always make implementation mistakes easy to spot.

9 Learned Strategies

The learned average strategies also give a qualitative sanity check. The following table shows selected information sets from one run. The exact numbers are not unique, and the Leduc information-set labels are longer because they include private rank, public rank, betting round, and public betting history. Still, the broad structure is sensible: weak unpaired hands are cautious, paired hands become aggressive, and facing a bet with a weak hand often leads to folding.

Card/public/history	Actions	First	Second	Third
J , no public, round 1 start	check/bet	0.927	0.073	–
J , no public, facing check-bet	fold/call/raise	0.927	0.054	0.020
J , public J , round 2 start	check/bet	0.341	0.659	–
J , public J , high-pot round 2 start	check/bet	0.018	0.982	–

The most useful feature of this table is not the exact decimal values, but the pattern. The same private J is weak before the public card is revealed, but becomes a strong paired hand when the public card is also J . The learned strategy responds to that change by betting much more often in the paired states.

10 Discussion and Bibliography

The main lesson from this project is that MCCFR is not a different conceptual algorithm from CFR. It is better understood as the same local counterfactual regret update with a sampled estimator for the values. External sampling is especially easy to understand because the update player still branches over all actions at visited information sets. Outcome sampling is the more purely Monte Carlo version: it samples a single trajectory and uses importance weights to correct the estimator. This connects the project directly to the importance-sampling part of the course.

The limitation of this project is that Leduc Poker is still small compared with realistic poker games. It is large enough to show why sampling helps, but not large enough to require abstraction. Natural next steps would be variance-reduction baselines for outcome sampling, larger Leduc variants, and abstraction methods such as bucketing.

External sampling is pedagogically attractive because it preserves the “compare all of my actions” step at each visited information set. This made the implementation close to the full-tree CFR baseline. The price is variance: a single sampled traversal sees only one chance outcome and one opponent action at each sampled opponent decision. Outcome sampling would go further by sampling actions for all players. That makes each iteration even cheaper, but the estimator is higher variance and more delicate to implement correctly. This tradeoff is exactly what appears in the experiment: outcome sampling visits the fewest nodes per iteration, but its exploitability curve is much noisier.

References

- [1] Seongwoo Hur. Study and implementation of regret matching and counterfactual regret minimization algorithms. GitHub repository, 2026. <https://github.com/swhur1214/cfr>.
- [2] Finnegan Southey, Michael P. Bowling, Bryce Larson, Carmelo Piccione, Neil Burch, Darse Billings, and Chris Rayner. Bayes’ bluff: Opponent modelling in poker. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, 2005. <https://arxiv.org/abs/1207.1411>.
- [3] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems 20*, 2007. <https://papers.nips.cc/paper/3306-regret-minimization-in-games-with-incomplete-information>.
- [4] Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. Monte Carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems 22*, 2009. <https://papers.nips.cc/paper/3713-monte-carlo-sampling-for-regret-minimization-in-extensive-games>.
- [5] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*, 1950.